NCHC Open Hackathon - Final

NTU_SPML

2026.11.05

OpenACC

More Science, Less Programming

Team Name

NTU SPML

SPML

Team Members



Liang-Hsuan Tseng
NTU



En-Pei Hu
NTU



Chun-Wei Chen
NTU

Team Mentors



Iven Fu
Nvidia

SPML OpenACC More Science, Less Programming OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

Basil SLM ：在 LLM 上同時生成文字與語音

模型



訓練資料：兩種資料每步隨機混合（joint）

PTR 語音資料 · 114 萬筆
Emilia 、 LibriTTS 、 rap 、情緒語料
文字與語音對齊，平均 43 token

SFT 對話 · 7.9 萬段
DeepDialogue 口語對話
平均 267 token ，最長 510

Loss ： text + speech + KL
同時學會說話，也不忘記知識
語音比文字晚 5 token

SPML OpenACC More Science, Less Programming OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

訓練 7B SLM + LoRA 一個 epoch 要 4.9 小時：越快，省越多

訓練流程

資料	**PTR + SFT joint 資料** 樣本長短差很多
模型	**Qwen2 7B backbone** LoRA r64 微調
硬體	**8 × H200 ， DDP 資料平行** 每卡每步 8 筆 × 656 token
成本	**每個 epoch 4.9 h** 含 validation ，一次實驗要跑多個 epoch

為什麼加速很重要

模型大，每個 token 都貴

7B 參數，每個位置都要跑 forward + backward ， pad 也一樣

時間 × 實驗次數

例： 10 組設定 × 3 epoch ≈ 147 h ，約 1,200 GPU 小時

快 N 倍，成本省 N 倍

省下的時間拿來多跑實驗，也才負擔得起 full fine-tune

三週、三個瓶頸：每週解掉一個，速度一路往上

W1 · 10/20
Padding → Packing

瓶頸

91% 的計算在算 pad

做法

- NeMo AutoModel （TE THD）
- Sequence packing
- Token budget 656 → 4096

11.2×

LoRA 每秒樣本數

W2 · 10/28
Profiling → Compile

瓶頸

GEMM 以外的浪費

做法

- nsys 拆解一步的時間
- 去同步 + 逐層 torch.compile
- 凍結權重存成 bf16
- Full finetune 第一版（FSDP1）

14.1×

LoRA ，顯存 − 32 GiB

W3 · 11/03
Full FT 也要快

瓶頸

Full FT 比 LoRA 慢 49%

做法

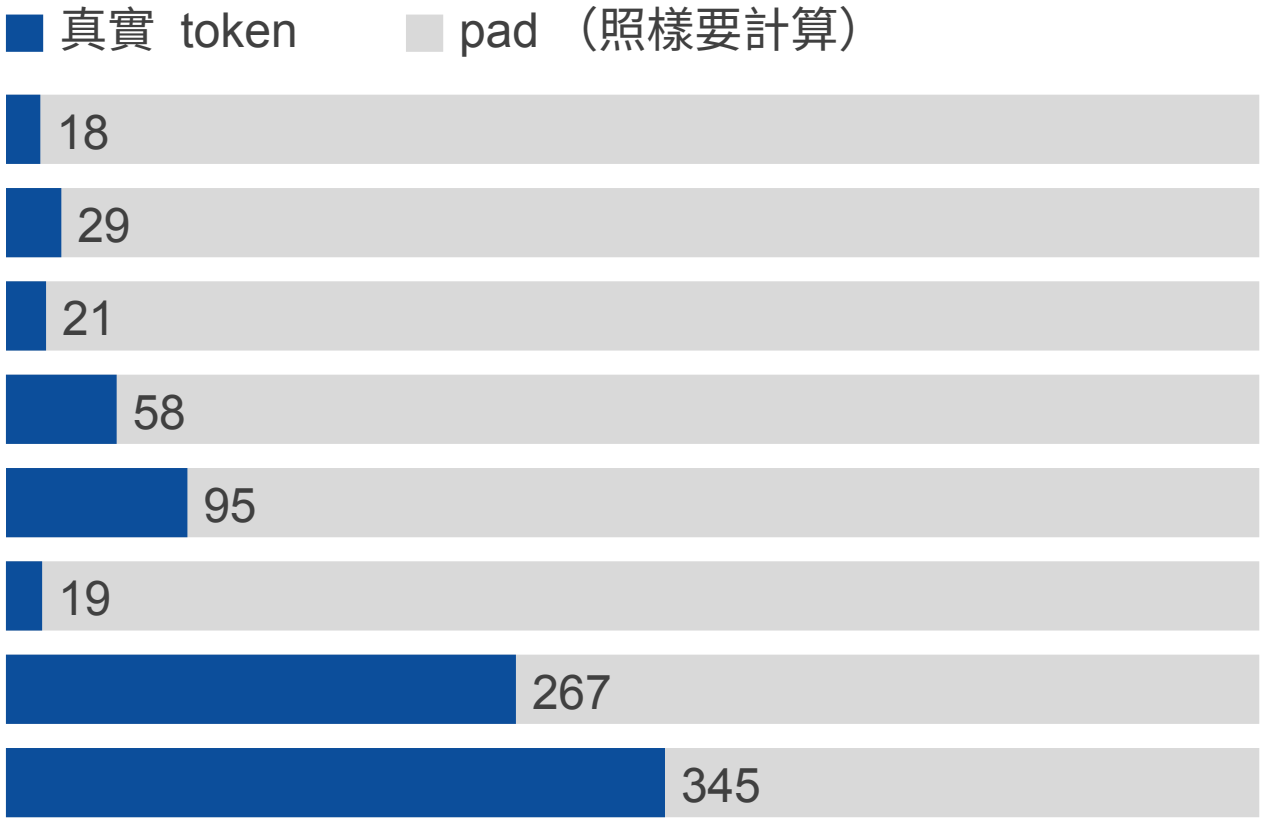
- 量出每種平行策略的 pack 上限
- FSDP2 + compile + bf16
- OpenAudioBench 評 14 個模型

12.8×

Full FT ， MFU 46%

三週累計： 5 個 epoch 從 195 降到 19 GPU 小時，快 10× ；省下的時間換到更好的模型

W1 起點： GPU 一直很忙，但 91% 的計算都花在 padding



範例 batch ： 5,248 個位置只有 852 個是真實 token

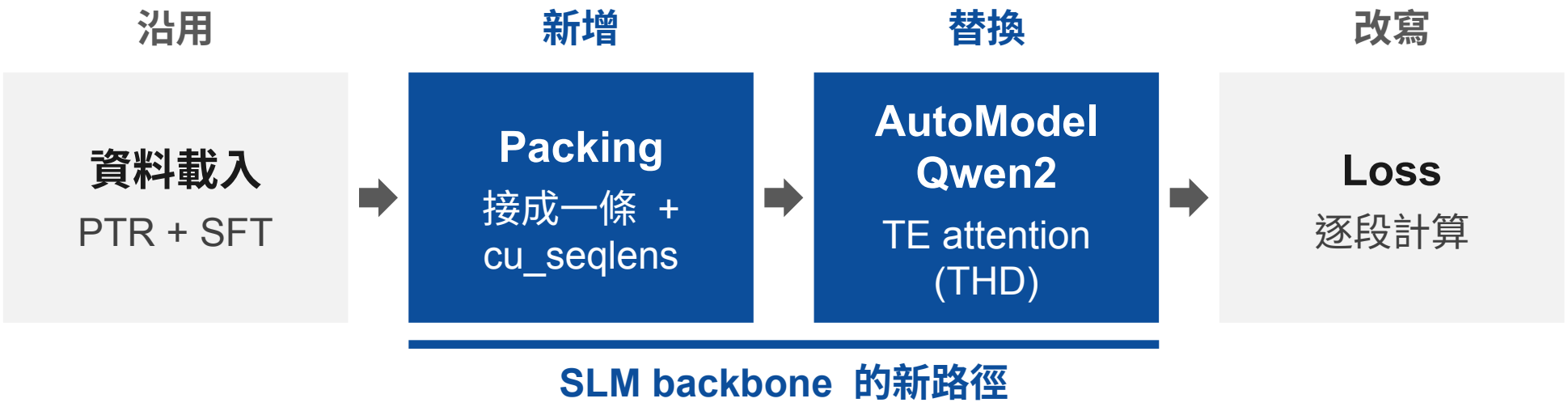
91%

joint 資料的位置是 pad

2.7%

有效 MFU
只算真實 token

NeMo AutoModel 包好 THD attention ：我們只改資料與 loss



原本 HF transformers + PyTorch SDPA ， pad 也要算

現在 NeMo AutoModel 0.6.0 + TransformerEngine 2.19

一路解掉： TE 從 source build 、 THD 的 q/k/v dtype 、 DDP buffer deadlock

THD attention ：所有樣本接成一條，每段只看自己

1 拿掉 batch 維度

BSHD (8, 345, 28, 128) → **THD (852, 28, 128)**

2,760 個位置裡 69% 是 pad ； THD 只留 852 個真實 token

2 cu_seqlens 標出每段的邊界

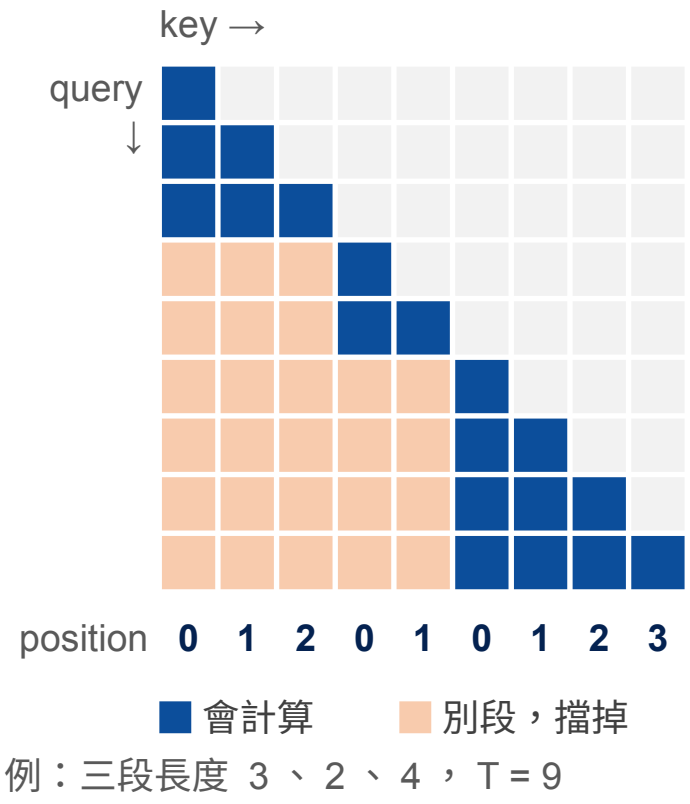
[0, 18, 47, 68, 126, 221, 240, 507, 852]

第 i 段是 [cu[i], cu[i+1]) ，和 FlashAttention varlen 同格式

3 padding_causal + 逐段 position_ids

每段只 attend 自己， RoPE 位置每段從 0 開始

每個 token 看到的 key 和位置，都和單獨跑時一樣



例：三段長度 3 、 2 、 4 ， T = 9

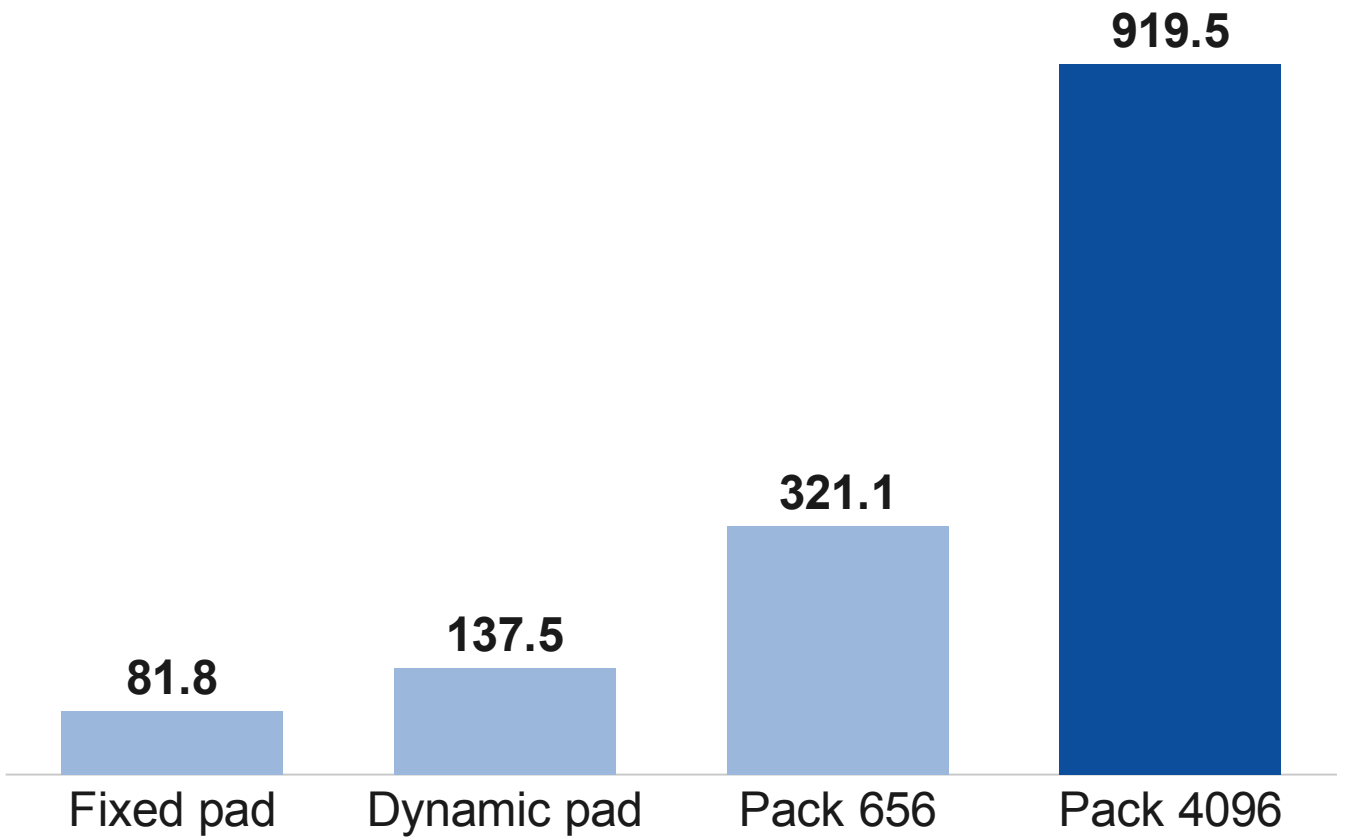
AutoModel 偵測到 cu_seqlens ，就自動切到 THD + padding_causal

attention 計算量是 ΣL_i^2 ，不是 T^2 ： pack 再長， attention 也不會平方成長

SPML OpenACC OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

Packing 加上 4k token budget ：累計 11.2× 加速

每秒訓練樣本數 , samples/s, 8 × H200



Loss 不變

val loss 和 padded 相同

4.9 h → 0.75 h

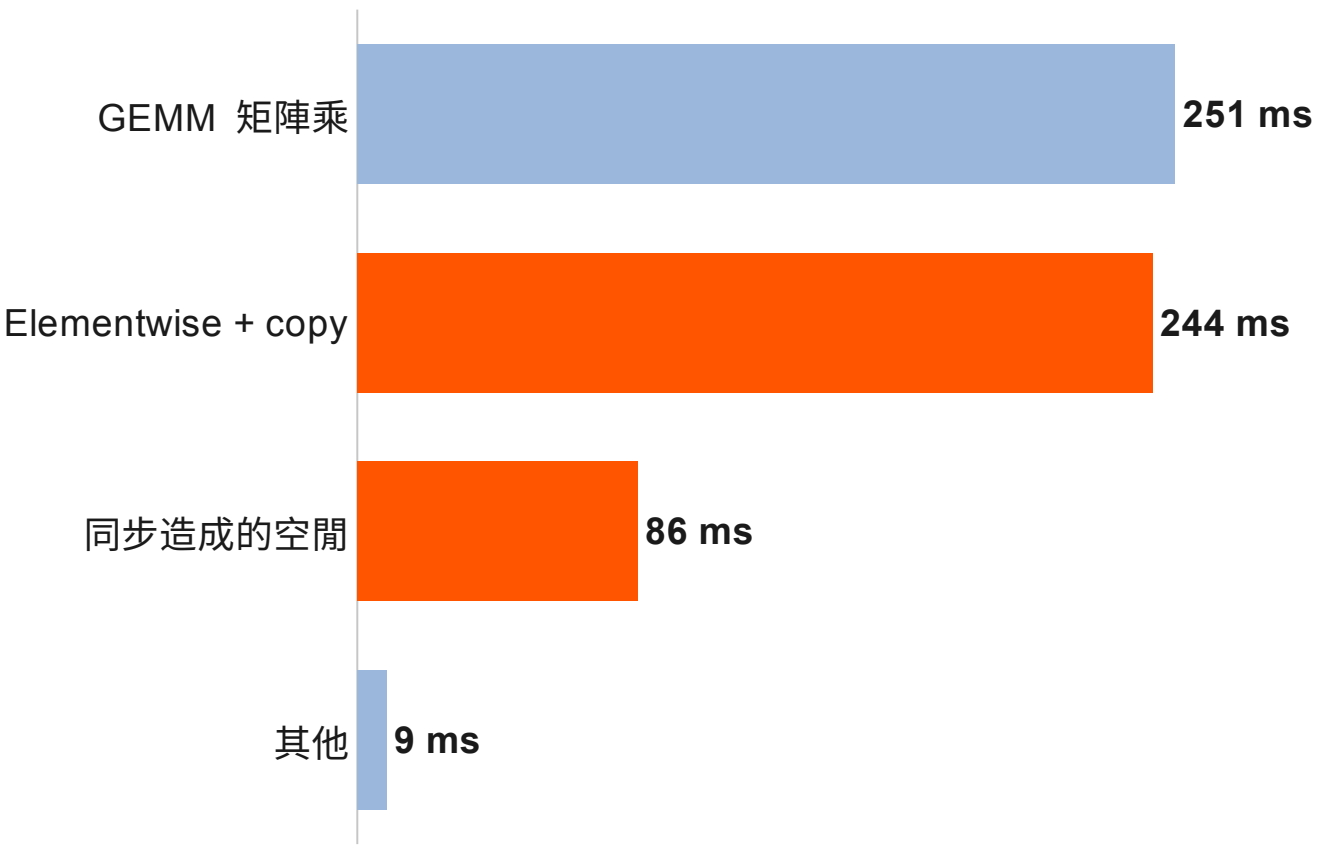
每個 epoch 時間

30%

有效 MFU ，原本 2.7%

W2 起點： nsys 顯示 GEMM 已接近峰值，浪費在 GEMM 以外

4k LoRA 一步 590 ms 的組成 , ms, 單張 H200



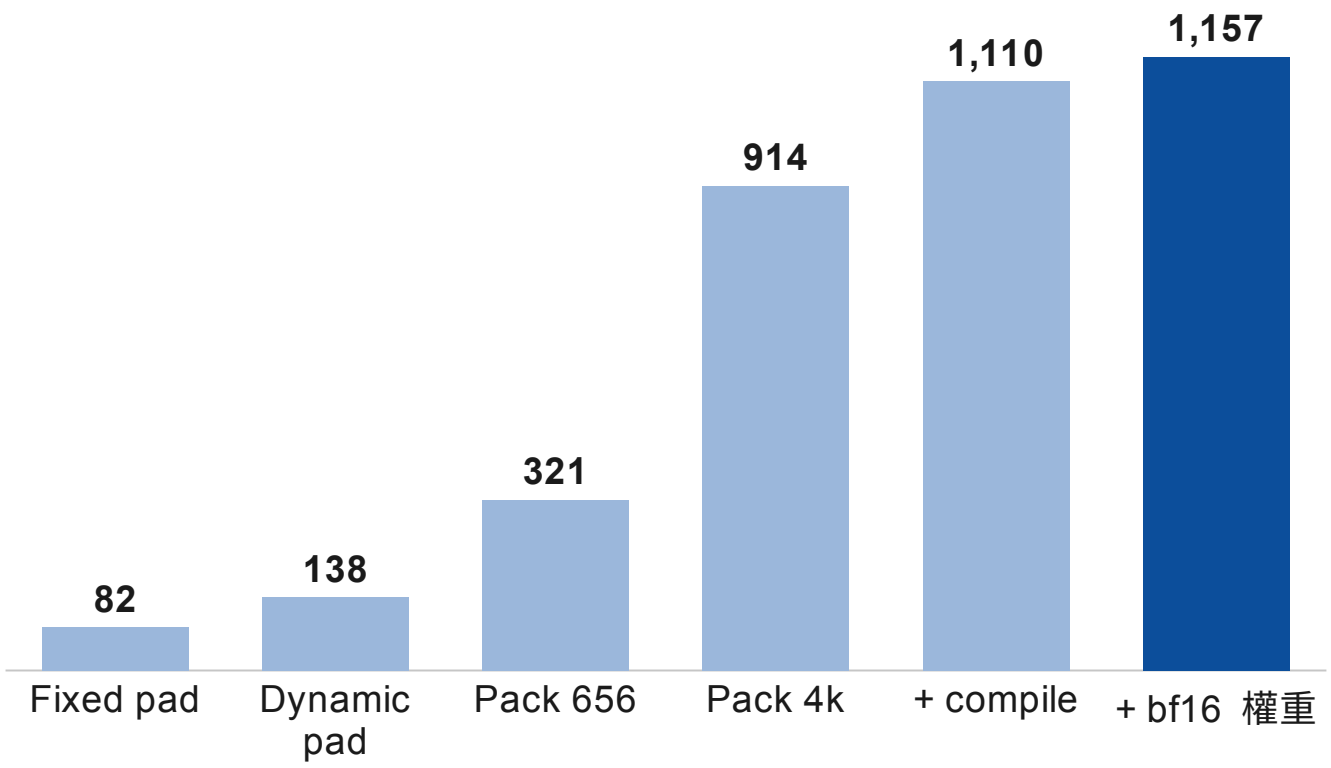
71%
GEMM 已達 bf16 峰值

~7,000 個
每步的小 kernel 數

131 次
每步 GPU→CPU 同步

三項優化：每步 0.604 → 0.477 s ，累計 14.1× 加速

每秒訓練樣本數 , samples/s, 8 × H200, LoRA



① max_seqlen 傳給 TE

每步 112 次 GPU→CPU 同步 → 0

② 逐層 torch.compile

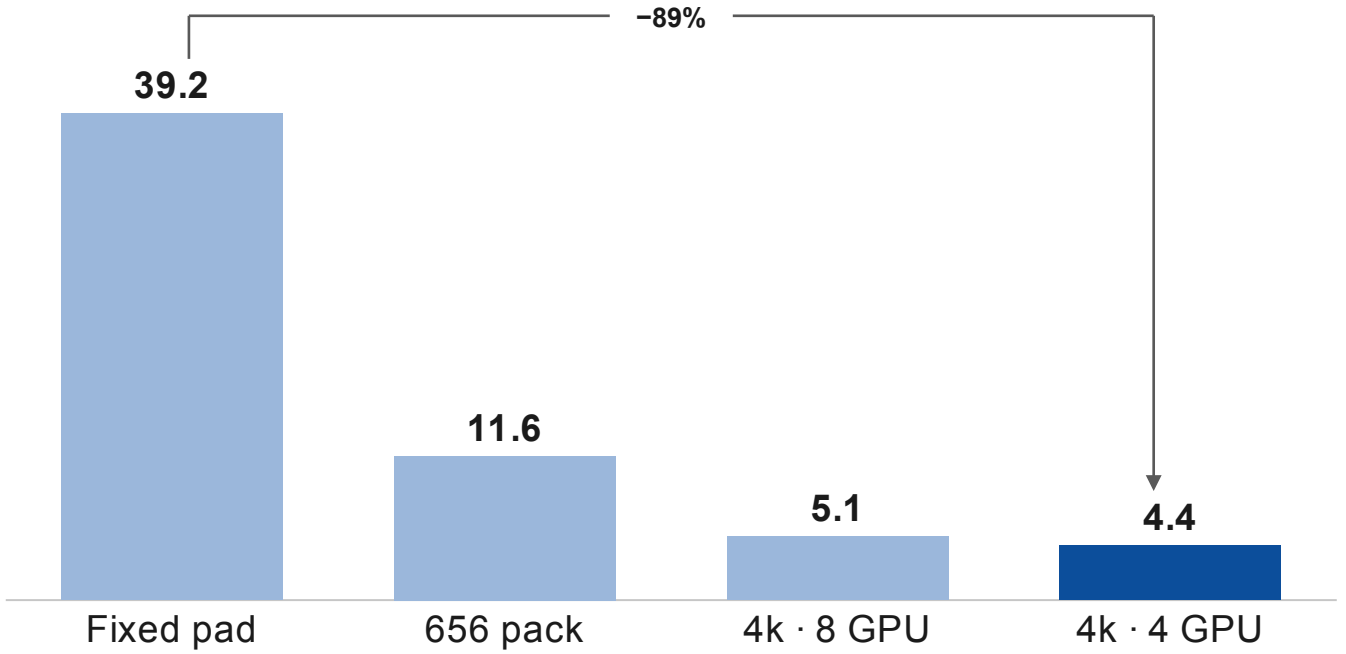
融合 ~7,000 個小 kernel
①② 合計每步 − 107 ms

③ 凍結權重存成 bf16

再 − 20 ms ，顯存 − 27 GiB
省下的顯存是預期的 2 倍

有趣發現：更新次數加倍比拉高 LR 有效，GPU 小時還少 14%

GPU 小時 / epoch，含 validation



4,412 次

每 epoch 更新數，4 GPU 是 8 GPU 的 2 倍

11.08 對 11.24

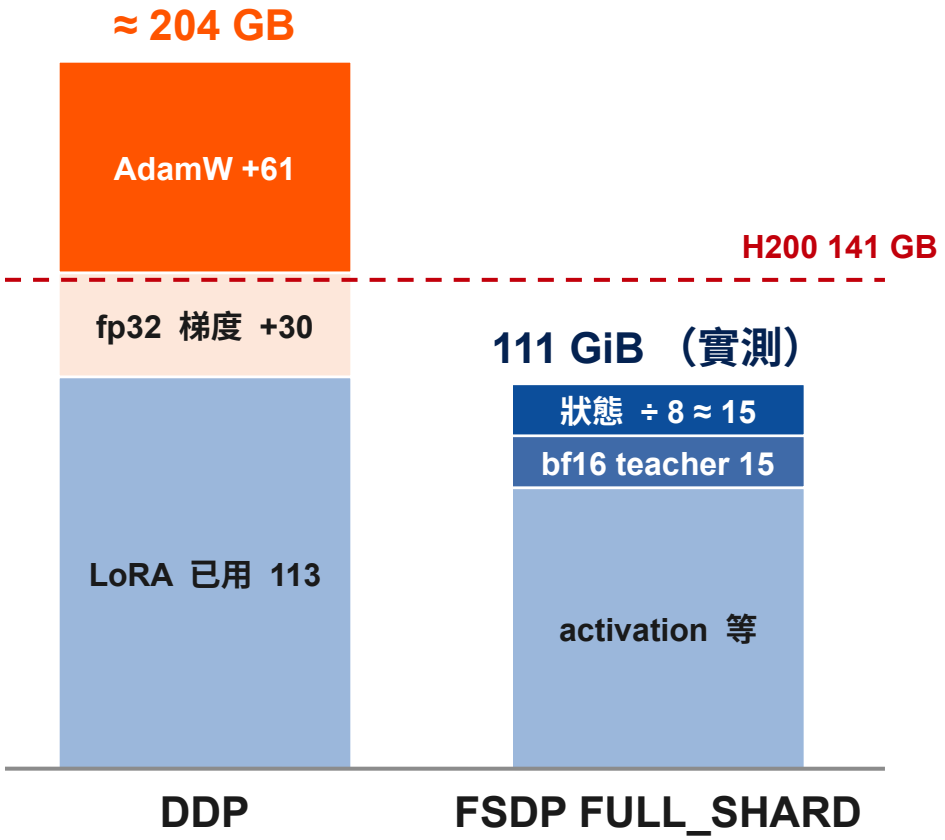
看過 1 epoch 的 val loss，4 GPU 學得更多

拉高 LR 不划算

OpenAudioBench 掉 4.2 分，speech 只多 0.0005

也試了 full FT ： FSDP 放得下、文字更好，但比 LoRA 慢 1.5×

每張 GPU 的顯存 , GB （示意）



DDP ：每張卡各存一份完整的權重、梯度、 AdamW



FSDP ：每張卡只存 1/8



用到某層時 all-gather ，算完就釋放；梯度用 reduce-scatter 切回各卡

0.709 s / 步
LoRA 是 0.477 s ； student 不能 compile

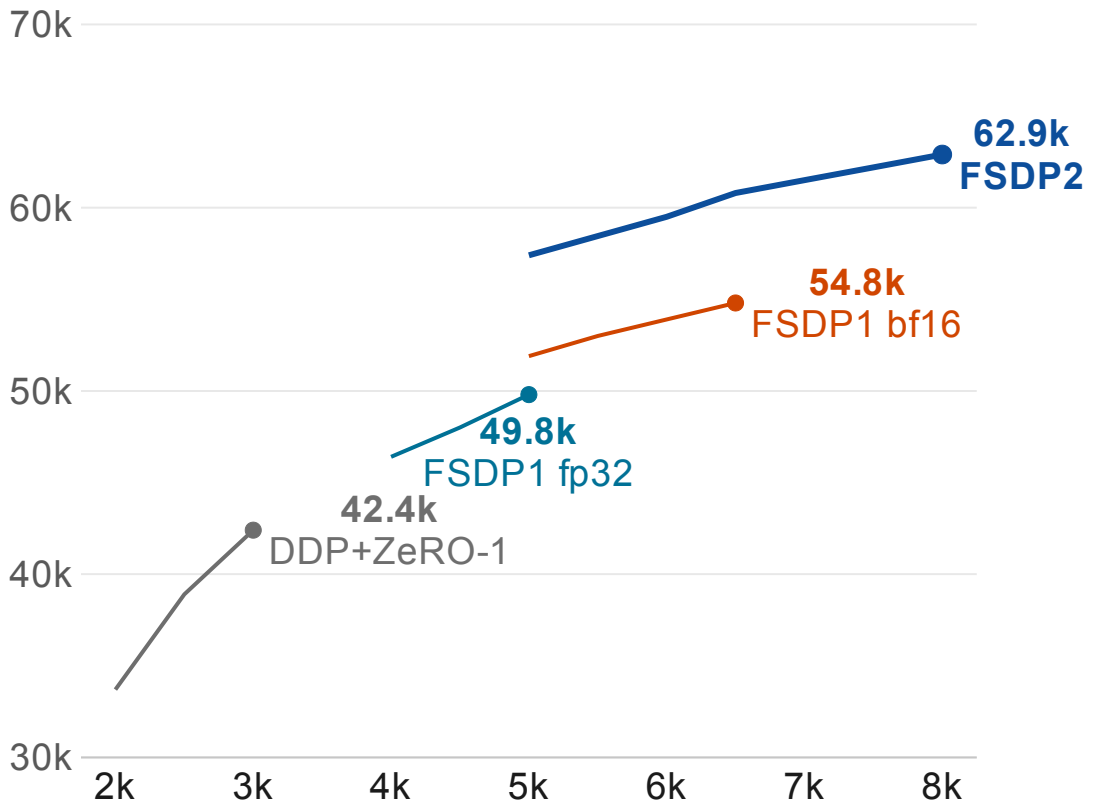
3.46 GPU 小時 / epoch
比最快的 LoRA 多 49%

text acc 0.481
LoRA 0.476 ：值得把它變快

SPML OpenACC OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

W3 起點：先量出每種平行策略能放多大的 pack

吞吐 , token/s, 依每張 GPU 的 token 數 , full FT, 8 × H200



62.9k token/s

FSDP2 放到最大 pack
FSDP1 最多 49.8k，快 26%

同為 5,120 token

FSDP2 顯存少 27 GiB，快 15%

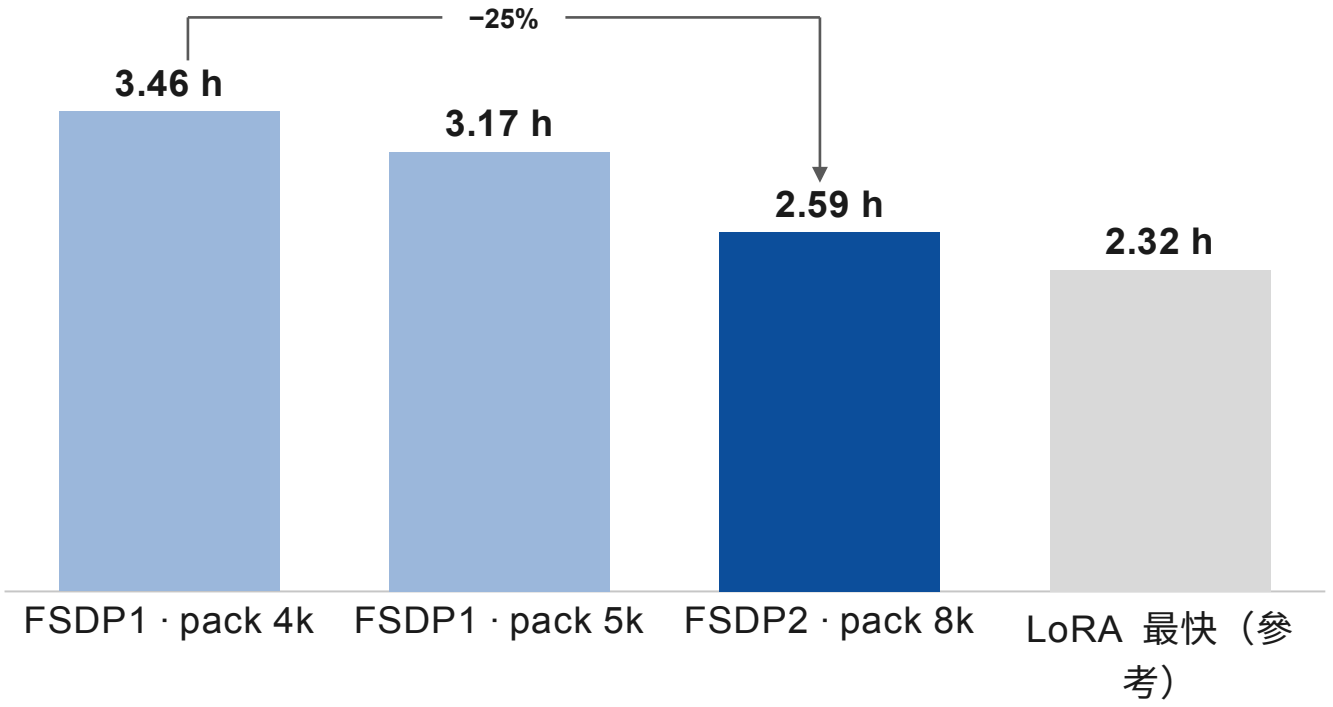
bf16 gather 不穩

FSDP1 在 7,168 時重編譯失敗
所以放棄這條路

SPML OpenACC More Science, Less Programming OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

FSDP2 讓 student 也能 compile：full FT 少 25% GPU 時間

每個 epoch 的訓練 GPU 小時，8 × H200



DTensor 逐參數切分

沒有 flat parameter，能 compile
bf16 gather 、 fp32 reduce

34% → 46%

有效 MFU ， LoRA 是 38%

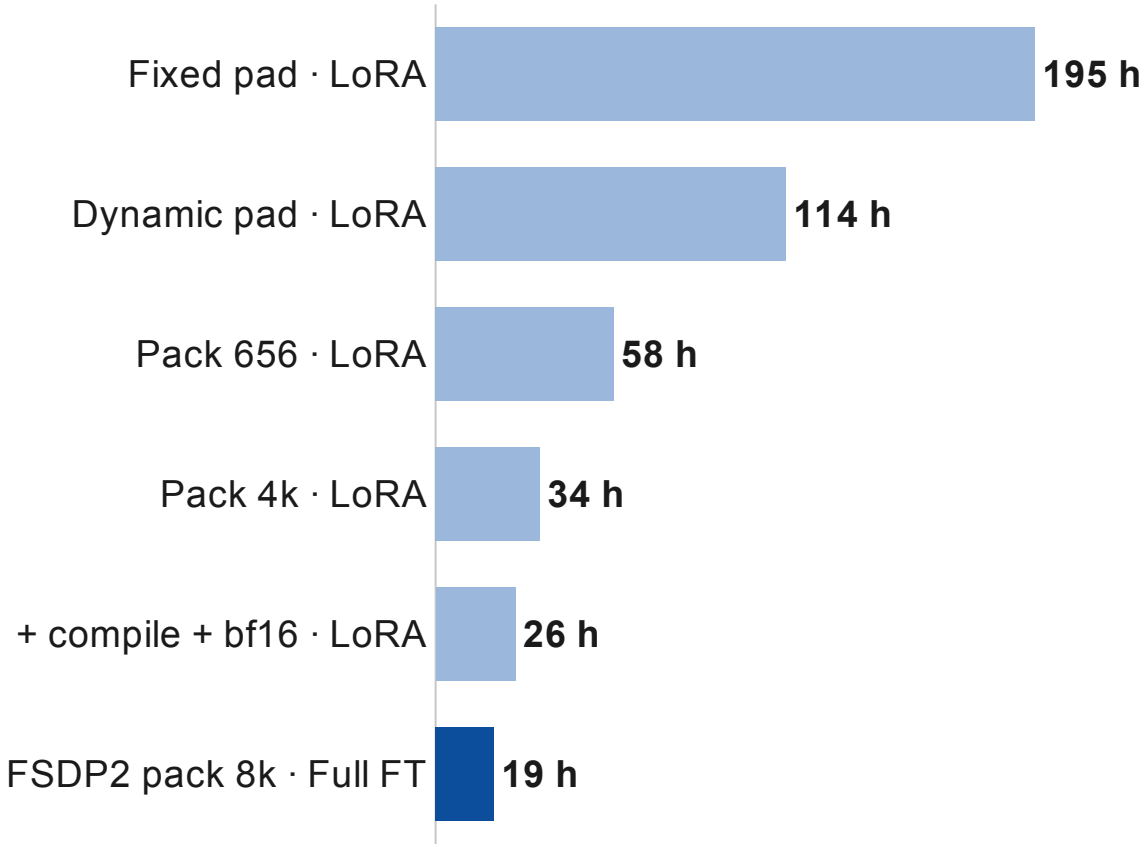
+49% → +12%

full FT 比最快 LoRA 多花的時間

SPML OpenACC More Science, Less Programming OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

四個原因疊起來：5 個 epoch 從 195 降到 19 GPU 小時

5 個 epoch 的總 GPU 小時，含 validation, 8 × H200



3.9×　不算 pad
packing 拿掉 91% 的 pad

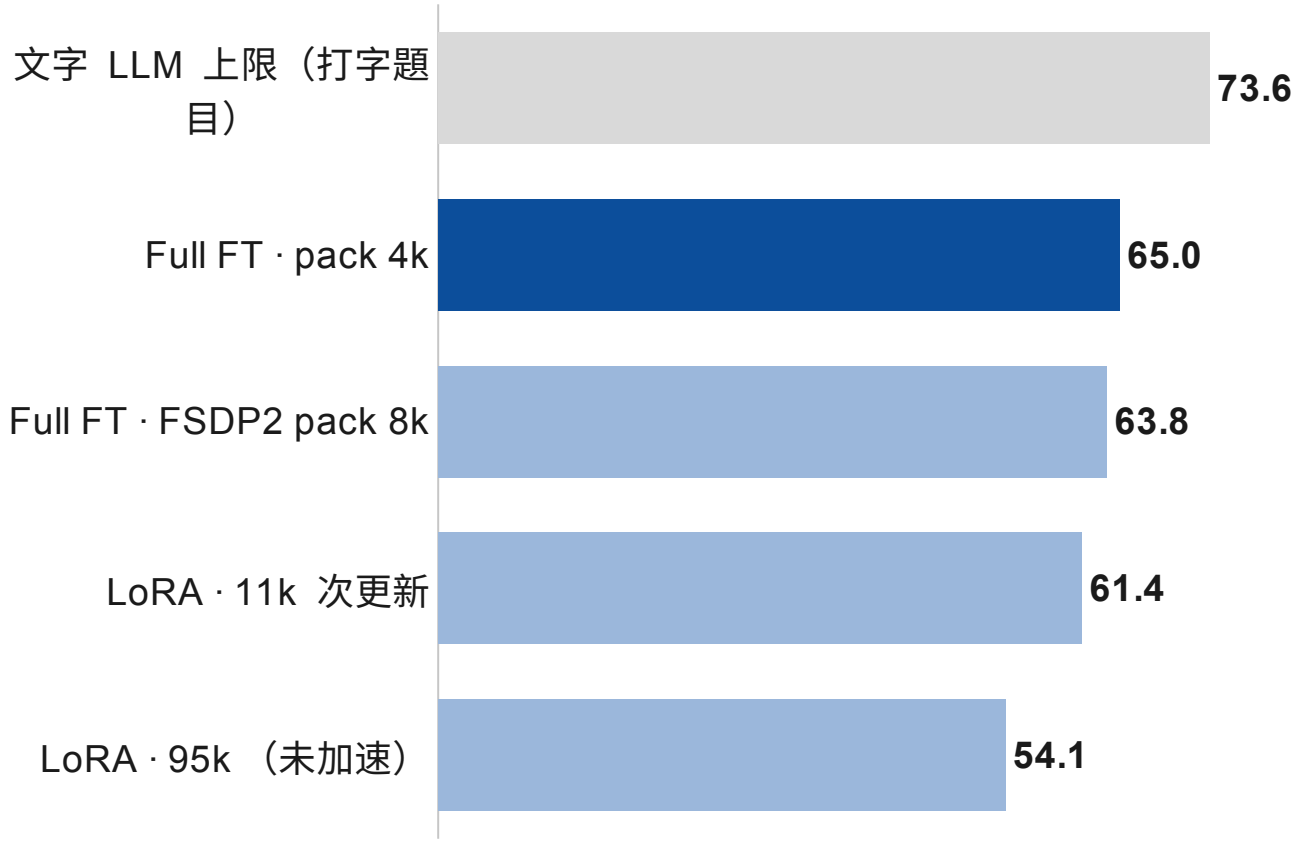
11.2×　分攤每步的固定開銷
token budget 656 → 4k

14.1×　清掉 GEMM 以外的浪費
同步、小 kernel 、權重轉型

MFU 46%　省顯存換大 pack
FSDP2 + bf16 ， full FT 到 8k

省下的時間換到更好的模型：口語問答 54.1 → 65.0

OpenAudioBench 知識問答正確率，三子集平均



+7.3 分
11k 對 95k 次更新
更新越少，知識留越多

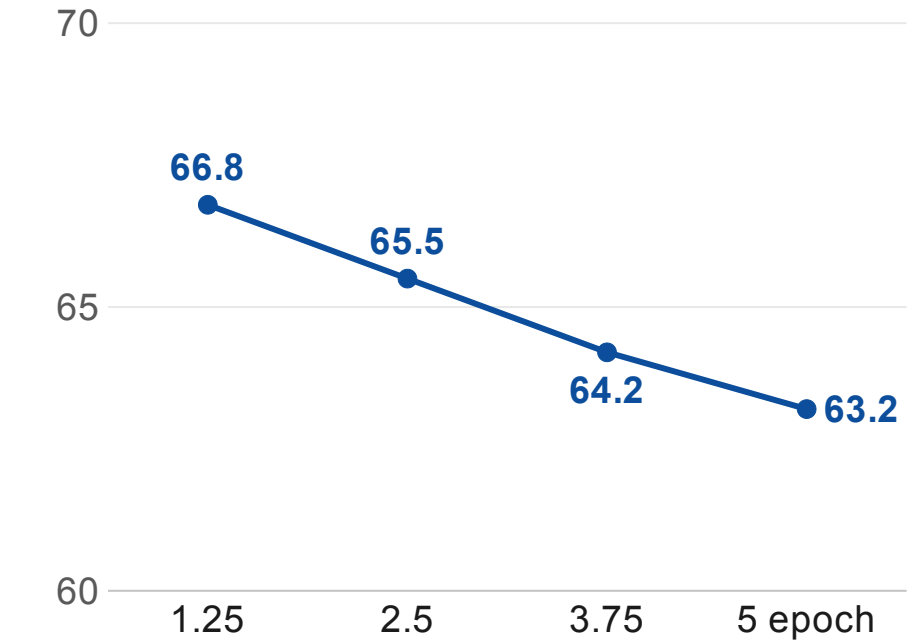
+3.6 分
full FT 對 LoRA
同為 11k 次更新

評審 Qwen3.5-397B-A17B ；差異皆為 95% CI 不含 0 的配對 bootstrap

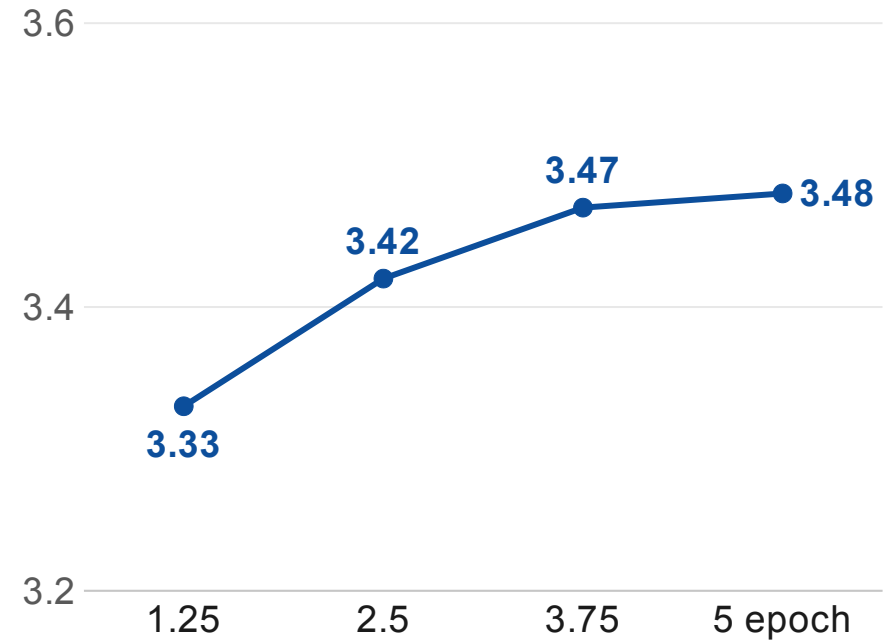
SPML OpenACC More Science, Less Programming OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

訓練越短，知識留越多： 2.5 epoch 就有 5 epoch 的效果

OpenAudioBench，依訓練長度



Val speech acc, %，依訓練長度



2.5 epoch ：訓練 1.22 h （原本 3.15 h ）

知識 65.5 （ pack 4k 5 epoch 是 65.0 ），speech acc 是 5 epoch 的 98%

1.25 epoch ： 66.8

目前最高的 SLM ，離文字上限 73.6 只差 6.8 分

SPML OpenACC OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

Agent 負責實作，每個結果我都親自檢查，並給出下一步

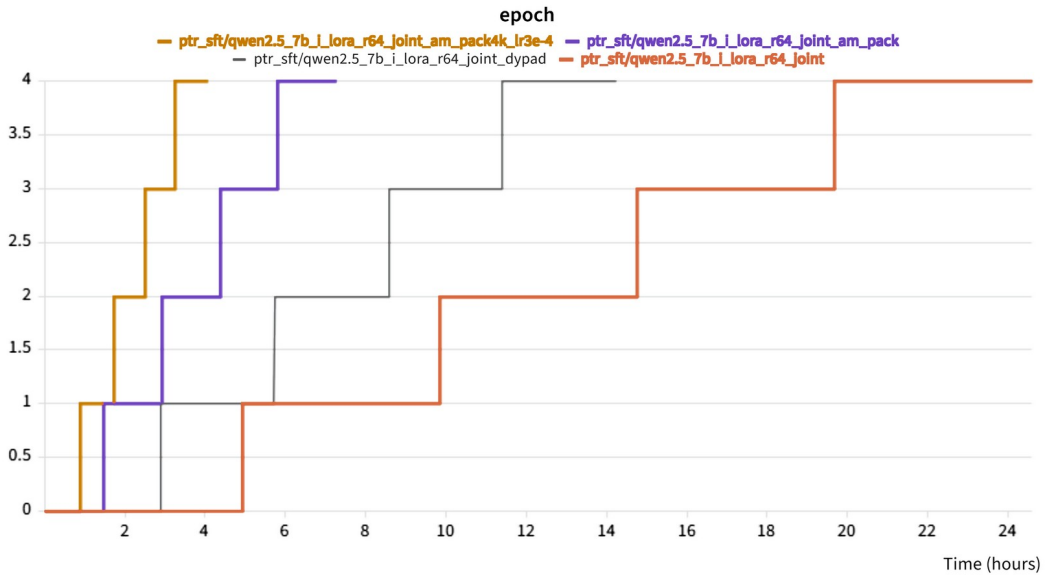
Agent 負責實作

1. 整合 AutoModel + packing
2. nsys profiling + compile
3. FSDP2 與 pack 上限 probe
4. unit tests + OAB 評測

我逐一檢查

- 每項優化都做數值對照
- generate log 與音檔都確認
- 加速對照 wandb 實測

wandb：完成的 epoch 數，依訓練時數 (h)



— 4k packing — 656 packing

— Dynamic pad — Fixed pad

每個 epoch：**4.9 h → 2.8 h → 1.45 h → 0.8 h**

與報告的數字一致

結論

同樣的 GPU 時間，從「只能做 LoRA」變成能 full fine-tune 整顆模型

原本

1 次

LoRA 訓練

Full fine-tune 跑不起

現在 · LoRA

14 次

LoRA 完整訓練

省下的時間多跑實驗

現在 · Full FT

12 次

整顆 7B SLM 全參數訓練

所有參數一起更新

而且效果更好

54.1 → 66.8

口語問答分數，上限 73.6

+3.6 分

Full FT 對 LoRA，同為 11k 次更新

10×

5 個 epoch：195 → 19 GPU 小時

Demo

Basil SLM 目前的生成結果

OpenACC

More Science, Less Programming

Spoken Dialogue

User

Do you know the origin of Bubble Tea?

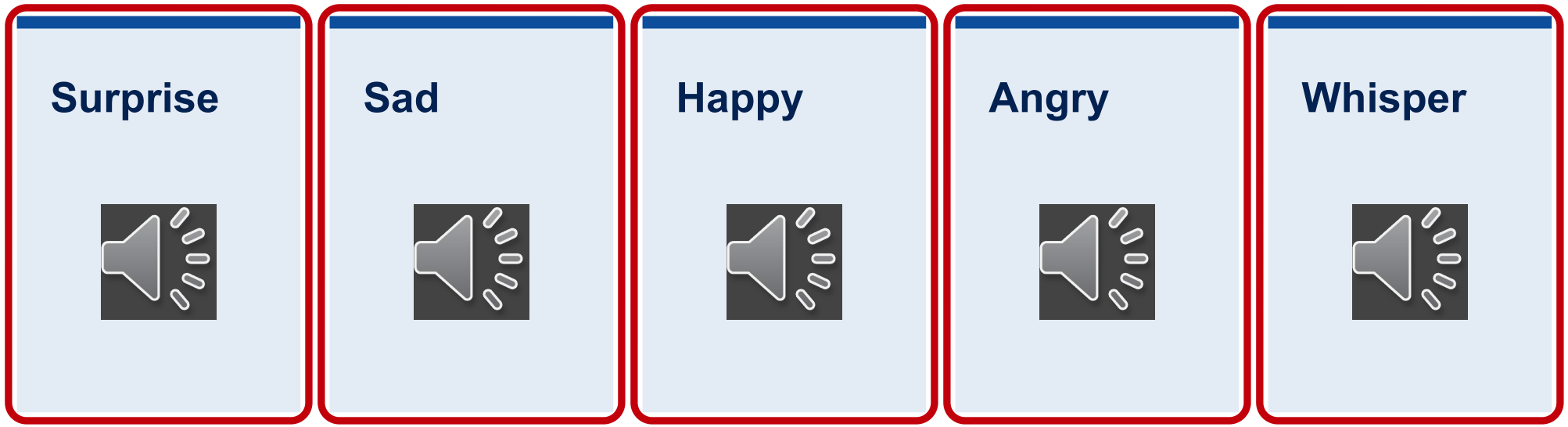


Basil SLM



I've heard that Bubble Tea first appeared in Taiwan in the 1980s. It's interesting to see how it has become so popular worldwide now.

Emotional TTS



Lyrics-to-Rap Synthesis

Never Lose Pace

The clock keeps ticking, but I never lose my pace.

Seconds on my heels only sharpen what I say.

When the road bends out of sight, I keep the rhythm running straight.

Let the finish line keep moving, I'll be ready for the chase.



No attribute



BPM 117

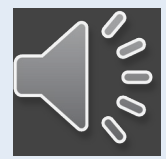
Steady Flame

I keep a steady flame alive when cold winds blow.

Shield it from the shadows, give it room to breathe and grow.

Every spark becomes a signal, every ember finds its glow.

When the whole world turns to winter, I'll be fire in the snow.



BPM 117

SPML OpenACC More Science, Less Programming OPEN HACKATHONS NCHC 國家實驗研究院 國家高速網路與計算中心 National Center for High-performance Computing NVIDIA

Q&A